2 main interfaces : + 2 main data structure approaches

1) ✓ Sequence   2) set   1) Arrays   2) Pointers.

↳ 1) static sequence interface

sequence of items $x_0, x_1, \ldots x_{n-1}$

build(x) : $O(n)$

len() : $O(1)$ → python stores size of collection in the memory by default

iter_seq() : $O(n)$

get_at(i) : $O(1)$ } as array address make it easy to access + word size is logn at least

set_at(i, x) : $O(1)$

which is [static arrays]

python has no static array

memory :

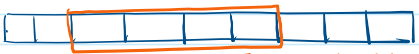
considering we have "word RAM" model

array is w-bit words + consecutive chunk

1 word size = logn at least why ?

as to perform getat & setat in constant time machine should be able to represent any index i within single machine word

so the word size in modern computer = 64 bits ~ $\log_2(6)$



array [i] = memory [address(0) + i]

hence make access $O(1)$

↳ 2) Dynamic sequence :

[static sequence] +

1) insert_at(i, x) → shift all items post (x at i th position / element) } $O(n)$

2) delete_at (i) → shift all to fill the space

3) insert/delete at first or last $O(1)$ for last as space always exist in python – else $O(1)$ for static array

$O(n)$ for first as data needs to shift

Dynamic Seq operations :

Static Array

1) Insert and delete cost $O(n)$ time

→ at last its $O(1)$ in python

else $O(n)$ in all cases

Linked list

Insert & delete at front at $O(1)$

[x |] just need head address change

else $O(n)$

get + set → $O(i)$ time

or $O(n)$ as each node should be checked

Bad at random access as the position needs to be travelled every time

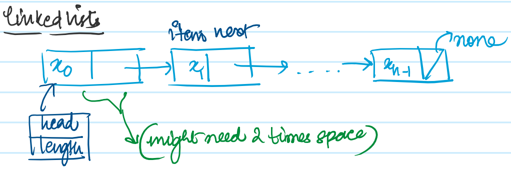
For last access save [head | len | tail]

Data Structure Augmentation

– add new info to make things better

None of the above work efficiently instead we use

Linked lists



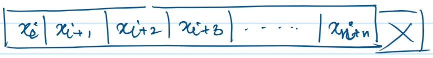
Solution → Dynamic array (Python lists)

→ Relaxes constraints on the size(array) = n (roughly n)

→ size of array $\Theta(n)$ & $\geq n$

→ maintain $A[i] = x_i$

→ if we visualize :



Array

length = n (of items stored)

size = (of the data structure Array with empty spaces)

① So assume we keep buffer & then extend it as the data grows!!

② But what if n = size?

allocate a new array of size := 2 × size

how will that help?

eg: we do insert_last() from empty array

pass 1 : [1 |] increase twice so size = 2

[1 | 2 | |] size = 4

> resize cost = $\Theta(1+2+4+8+16+\ldots+n)$